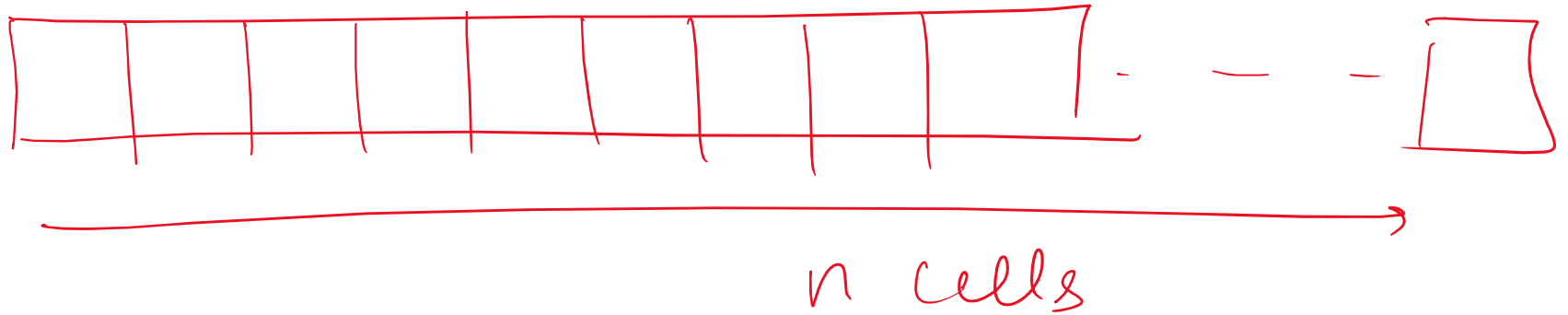


The game of Snakes and Ladders has a board with n cells where you seek to travel from cell 1 to cell n . To move, a player throws a six-sided dice to determine how many cells forward they move. The board also contains snakes and ladders that connect certain pairs of cells. A player who lands on the mouth of a snake immediately falls back down to the cell at the other end. A player who lands on the base of a ladder immediately travels up to the cell at the top of the ladder. Suppose you have rigged the dice to give you full control of the number for each roll. Design an efficient algorithm to find the minimum number of dice throws to win.

visited array (n)

move array $\rightarrow$ you store the mapping of every
(n) cell

move [i] = j ($i+1$)



this is now you have.

for each cell you have 6 options:

$$\begin{pmatrix} i+1 & i+3 & i+5 \\ i+2 & i+4 & i+6 \end{pmatrix}$$

you perform a bfs for each step

a every i^{th} cell if

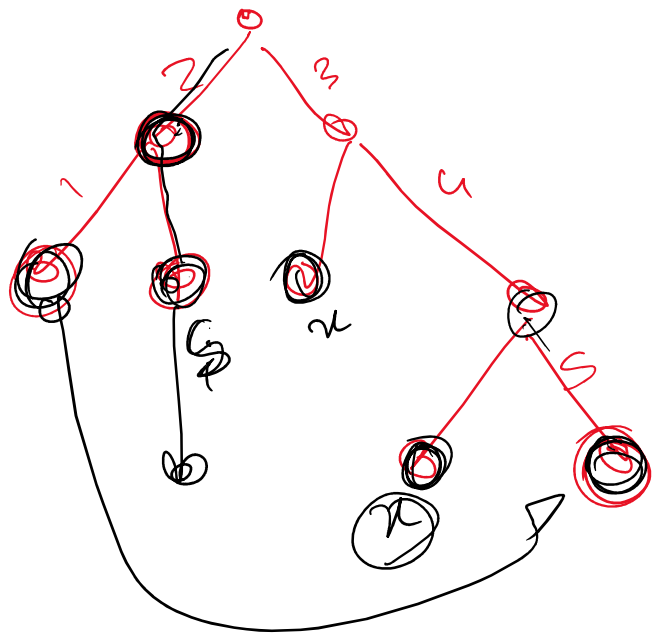
$$n - i \geq 6$$

we would always have 6 options.

T.C. $\Rightarrow$

$$O(6n) \sim O(n)$$

The diameter of a tree is defined to be the maximum distance between any pair of vertices.
 Design a linear time algorithm to determine the diameter of a tree. Formally prove correctness of your algorithm.



diameter

u, v

BFS



Proof

Given Graph $G:(V, E)$

Claim: v is one of the endpoints of a possible diameter

Proof Let v not be an endpoint ~~to~~

$\exists (a, b)$, $v \neq a$ nor $v \neq b$

$\Rightarrow$ where (a, b) is a diameter

$$d(s, v) \geq d(s, a)$$

$$d(s, u) + d(u, v) \geq d(s, u) + d(u, a)$$

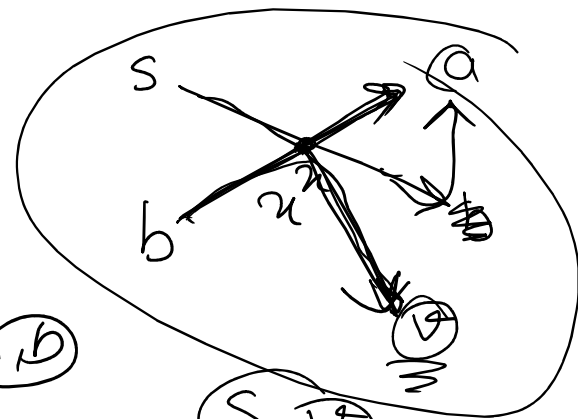
$$\Rightarrow d(u, v) \geq d(u, a)$$

$$v \rightarrow a$$

$$b \rightarrow a$$

$$(u, v)$$

$$(a, b)$$



$$(s, v)$$

$$(s, a)$$

$$(s, b)$$

$$\sqrt{v} \rightarrow u$$

$$v \rightarrow a \Rightarrow$$

$$b \rightarrow a \Rightarrow$$

$$(u)$$

$$(v \in V)$$

$$u = a$$

$$u = b$$

This contradicts the fact that a, b is a diameter
as $d(b, v) > d(b, a)$

$\therefore$ By contradiction v is one of the end pts

$$d(b, v) > d(b, a)$$



$$d(b, u) + d(u, v) > d(b, u) + d(u, a)$$

Three Seashells is a solitaire game, played on a connected undirected graph $G = (V, E)$. Initially, three tokens are placed on distinct start vertices $a, b, c \in V$. In each turn, you must move all three tokens, by moving each token along an edge in E from its current vertex to an adjacent vertex. At the end of each turn, the three tokens must lie on three different vertices. Your goal is to move the tokens onto three goal vertices $x, y, z \in V$. Describe and analyze an algorithm to determine whether a given Three Seashells puzzle is solvable. Your input consists of the graph G , the start vertices a, b , and c , and the goal vertices x, y , and z . Your output is either TRUE or FALSE.

① a

② b

③ c

$G = (V, E)$

(a, b, c)



(x, y, z)

(y, x, z)

(z, y, x)

(a, b, c)

(x, y, z)

(a, b, c)



(f, e, d)



$s_0 = (a, b, c)$

visited

queue

if queue empty → enqueue(s_0)

for all neighbors of a :

for all neighbors of b :

for all neighbors of c :

curr = dequeue()

if curr = perm(x, y, z):

return true

state(a', b', c') → pairwise distinct

if state not visited:

visited
enqueue.

return false.

(H)

✓
(a, b, c) → (a', b', c')

T.O.C

=

$$n \left(n(n-1)(n-2) = O(n^3) \right)$$

$n \rightarrow \text{deg}$ 2) $(n-1)$

$O(n^3)$

$(n^3 \cdot O^3)$

$\Rightarrow O(n^6)$

Time = state x work per state

$$\left((n-1)(n-1)(n-1) \right)$$

